

INDIAN INSTITUTE OF TECHNOLOGY DELHI

COL764: Information Retrieval and Web Search

COL 764 Assignment 1: Sparse Retrieval

Submitted by

Aditya Anand

Instructor

Prof. Srikanta Bedathur

November 7, 2026

The submitted system is a C++ engine behind the three required Python entrypoints in `submission/retrieve.py`. All indexing, compression and scoring happen in C++; Python only dispatches, picks parameters from collection statistics, and gzips the finished index. A Python path (`indexer.py`, `boolean_vsm.py`, `bm25.py`, `custom_scorer.py`) is submitted alongside it, since the Boolean and vector-space components are graded on independent correctness and a small readable implementation is far easier to unit-test than the packed postings reader.

1 Indexing design

1.1 Analysis chain

Text is lowercased through a 256 entry byte table and split at every non alphanumeric character. Single letters separated only by periods are then glued back together, turning `U.S.` into `us` and `F.B.I.` into `fbi`; without that rule an acronym decomposes into three length-1 tokens, and length-1 tokens are discarded. Stopping uses `INDEX_STOPWORDS`, a Lucene-style English list extended with question words, applied identically at index and query time. A second, much more aggressive Fox-style list (`STOPWORDS`) is never used for indexing; it only prunes the RM3 forward index on collections of at least 20,000 documents, where common verbs would otherwise dominate the expansion candidates. Stemming is Porter by default; `IR_STEMMER=1` or `0` selects the S-stemmer or no stemming. The stemmer and stopword flags are stored in `meta.bin` and restored in `load_index`.

Boolean, VSM and Python BM25 use a simpler indexer (`indexer.py`: S-stemming, no stop list, no acronym folding). Table 2 therefore compares two analysis chains on the same topics.

1.2 What is persisted

Five files reconstruct the scorer. Four are gzipped after the build; `meta.bin` stays uncompressed so `load_index` can read the analysis flags before inflating the rest.

- `dict.bin` is the sorted vocabulary, front-coded so shared prefixes are not repeated. Term lookup is a binary search.
- `postings.bin` uses gap-plus-tf vbyte on short lists ($df<8$) and Simple-9 plus Elias–Fano on longer ones. High-df query terms dominated latency; Elias–Fano decodes them faster than vbyte. Title frequencies sit in a small sidecar.
- `docmap.bin` stores the external document ids, packed when they share a length and a small alphabet, otherwise raw or front-coded.
- `doclens.bin` stores N, avgdl, and one byte per document for $|d|$/avgdl. BM25 only needs that ratio roughly, so a float per document would waste space.

1.3 Build path

`build_index_from_file` `mmap`s the JSONL file and runs SPIMI over line spans. At most four workers each own a contiguous range of document ids. The count drops at 400k and 800k documents so the mapping and per-worker term maps stay inside 8 GB. Vocabulary ranges merge and encode in parallel, but dictionary front-coding stays serial, so W=1 and W=4 write the same bytes.

2 Retrieval models

2.1 Boolean and vector space

`boolean_search` intersects or unions posting sets and returns unranked, sorted doc ids. Table 1 is the set-level comparison the assignment asks for; nDCG is omitted because the output is not a ranking. AND is too strict for these topics and OR is almost the whole collection.

Table 1: Boolean AND/OR on the 50 CORD-19 dev topics, rel > 0. **Python** is `boolean_search` (S-stem, no stops). **C++** is the same set operations on postings built with `tokenize()` (Porter + stops). Means are macro-averaged over all 50 topics: an empty AND set contributes 0 to Prec./Rec./F1, so Python AND Prec. 0.1165 is not 0.36 over the 16 non-empty queries. F1 is per-query then averaged.

	Mode	mean $\lvert R\rvert$	Prec.	Rec.	F1	# empty
Python	AND	3.7	0.1165	0.0037	0.0066	34 / 50
Python	OR	1.505×10^5	0.0032	0.9853	0.0065	0 / 50
C++	AND	134.6	0.3143	0.0595	0.0734	15 / 50
C++	OR	8.14×10^4	0.0061	0.9434	0.0121	0 / 50

Python: `indexer.py` then `boolean_search`. C++: no Boolean API, so `tokenize()` on the corpus, in-memory postings, then the same AND/OR on query tokens. Boolean AND/OR and `lnc.ltc` cosine are checked against hand-computed output on a tiny corpus in `tests/test_retrievers.py`.

`vsm_score` is `lnc.ltc` cosine, $w = (1+\log tf)\cdot\ln(N/df)$ on both sides. $\|d\|$ is the L2 norm of the *full* document vector, stored at build time: a query-term subspace norm would score every hit 1.0 on a one-term query. Ties break by score descending, then doc id ascending, throughout.

2.2 BM25+

`bm25.py` and the C++ loop share the Okapi body; δ=0 is textbook BM25. C++ also uses query-tf normalisation and a title indicator:

$$w_q \left[\frac{tf_{\text{eff}}(k_1+1)}{tf_{\text{eff}} + k_1(1-b+b\,|d|/\text{avgdl})} + \delta + \beta \mathbb{1}[tf^{\text{title}} > 0] \right], \quad w_q = \frac{qtf_q}{\sum qtf} \ln\!\left(1 + \frac{N - df + 0.5}{df + 0.5}\right).$$

Shipped w_{title}=0, so tf_{eff} is body tf and the title enters only as β. The title window is the first 12 analysed token positions (a stopword still occupies a slot, so the window does not drift with the stop list). Scoring is term-at-a-time into a generation-stamped accumulator; the top k come from `nth_element` then a sort. The BM25 length factor is a 256-entry table indexed by the quantised $|d|$/avgdl byte.

2.3 Custom scorer and RM3

`custom_scorer.py` min–max fuses Python BM25 (k_1=0.9, b=0.4) and cosine at 0.85/0.15 over a pool of 200 (raw mix is invalid: BM25 is unbounded). It loses to Python BM25 on CORD-19 (Table 2), so `retrieve()` does not call it.

Competition ranking is C++ BM25+ with optional RM3. From the top *fb_docs* of the first pass, $P(w|R) = \sum_i P(d_i|R)\,(tf_{w,d_i} + \mu P(w|C))/(|d_i| + \mu)$ is IDF-weighted and mixed as $\alpha \cdot \text{original} + (1-\alpha) \cdot \text{expansion}$. The document prior is $P(d|R) = \lambda\,\text{uniform} + (1-\lambda)\,\text{log1p(score)}$: λ=1 on newswire with a full second BM25 pass, λ=0.25 on abstracts with a rerank of at most 250, so expansion can still promote a newswire document that missed the first pool. Expansion is skipped if feedback-score variance is below 10^{-6}, or if a candidate appears in fewer than two feedback documents when *fb_docs* $\geq$ 12. Counts come from a per-document CSR of $tf \cdot \text{idf}$, df-banded and capped.

2.4 Collection-adaptive parameters

`adapt_params`(N, avgdl) sets the knobs from collection shape only. It interpolates short-document → newswire on analysed avgdl $\in [120, 155]$, then applies discrete regimes (near-duplicate titles, short answers, forums, large abstracts, small full-text, newswire). C++ `retrieve` then turns RM3, δ and the title bonus off on document-as-query inputs and tilts α on very short or long queries; `retrieve_bm25` sets `apply_qlen_heuristics=false` so ablations stay BM25+.

Cuts are on (N, avgdl), not a named collection id; an unseen pair matching no branch keeps the blend with RM3 off. Lying to `adapt_params` on CORD-19 (N=171,332, avgdl=109.7): ±15% stays in the large-abstract regime (nDCG@10 0.7458, or 0.7317 once +15% avgdl starts the b-blend). Crossing a cut (avgdl=136 or N=99,999) drops to the blend with RM3 off at 0.69, still above textbook BM25 (0.636). Table 4 is the same check on real collections, which is why RM3 stays off on Touché-length forums.

3 Evaluation

The numbers below come from the final submission code, re-run with `harness/metrics.py` (graded nDCG with $2^{\text{rel}} - 1$ gain; MAP@10, MRR and P@10 binarise rel > 0) on the 50 released TREC-COVID topics, N=171,332 and analysed avgdl=109.71. Held-out topics were never used for tuning. C++ rows share the Porter and stopword chain; Boolean, VSM and Python-BM25 rows use the S-stem indexer.

Table 2: Ranked retrievers on CORD-19 dev, k=10. Boolean AND/OR is unranked; set metrics are in Table 1. Boolean, VSM and BM25 correctness is in `tests/test_retrievers.py`.

Run	nDCG@10	MAP@10	MRR	P@10
TF–IDF cosine VSM (`lnc.ltc`, Python)	0.2815	0.0067	0.5178	0.3160
Python BM25, k_1=1.2, b=0.75, δ=0	0.5714	0.0139	0.8393	0.6360
Python BM25⊕VSM fusion (0.85/0.15)	0.5514	0.0129	0.7992	0.6300
C++ BM25, k_1=1.2, b=0.75, δ=0	0.6356	0.0155	0.8875	0.6920
C++ BM25+ (k_1=2.15, b=0.60, δ=0.80), no title	0.6323	0.0151	0.8990	0.6900
+ title β=0.60	0.6815	0.0171	0.9283	0.7380
+ RM3 (α=0.62, μ=9000, 30/40)	0.7458	0.0198	0.9589	0.8260

Two rows there deserve attention. The gap between Python BM25 at 0.5714 and C++ BM25 at 0.6356 under identical k_1, b and δ is purely the analysis chain, since the formula is the same: Porter plus a stop list is worth about 0.064 nDCG@10 here. And BM25+ without a title is very slightly *below* plain BM25 (0.6356 → 0.6323), the first sign that the (k_1, b, δ) move only pays once the title indicator is present. That 0.6323 row is the b=0.60 point of Figure 1.

The efficiency gain is codec work plus the four-worker build: the index shrank from 41.2 MB to 16.6 MB, mean latency from 2.76 ms to 2.28 ms, and build from 13.15 s to 2.20 s, while nDCG@10 rose from 0.7345 to 0.7458 and P@10 from 0.7980 to 0.8260. Simple-9 plus Elias–Fano was chosen because vbyte everywhere would have been marginally smaller on some lists and much slower on the ones that matter.

From the initial submission. The first zip was already C++ BM25+ with RM3, but in a single source file (`cpp_engine.cpp`). It used serial SPIMI, gap-plus-vbyte postings with skip tables, a persisted `fwd.bin`, no gzip, and one frozen PARAMS block (k_1=2.1, b=0.6, δ=0.25, β=0.75, μ=1000, α=0.7, 30/20). The competition round

Table 3: CORD-19 through the two-process harness on the 50 public topics. MAP@10 is small because these topics have hundreds of relevant documents each. Row 1 is a re-run of the initial-submission C++ engine (single file, vbyte, no gzip). Row 2 is this entry.

Phase	nDCG@10	MAP@10	MRR	P@10	Build	Latency	Index
Initial submission	0.7345	0.0189	0.9240	0.7980	13.15 s	2.76 ms	41.2 MB
This entry	0.7458	0.0198	0.9589	0.8260	2.20 s	2.28 ms	16.6 MB

split that file into `analysis`, `codecs`, `corpus_io` and `engine.h`, then:

- **Build:** up to four workers (8 GB cap at 400k/800k docs), mmap JSONL, byte-identical output for W=1 and W=4.
- **Disk:** gzip after build; RM3 CSR rebuilt at load (no `fwd.bin`); long lists Simple-9 + Elias–Fano; short lists stay vbyte; packed `docmap.bin`.
- **Analysis:** dotted initials (`U.S.`→`us`); leftover length-1 tokens dropped (the first zip kept them).
- **Scoring:** query-tf normalisation, 256-entry length table, generation-stamped accumulator; news vs abstract RM3 (full second pass vs rerank-250); query-length overlay.
- **Knobs:** `adapt_params`(N, avgdl) instead of one CORD point. Large abstracts now use k_1=2.15, δ=0.80, μ=9000, α=0.62.

Table 4: The same C++ engine on every local collection. **BM25** is textbook Okapi (k_1=1.2, b=0.75, δ=0, no title, no RM3) through `retrieve_bm25_only`. **Ours** is the competition `retrieve()` with `adapt_params`. BEIR test splits; Antique test; Vaswani.

Collection	N	avgdl	#q	nDCG@10		Ours		RM3
				BM25	Ours	MAP@10	P@10	
Toy	20	7.8	6	0.8793	0.8793	0.8250	0.317	off
Cranfield	1 400	100.8	225	0.2922	0.3308	0.2891	0.271	on
CACM	3 204	33.7	52	0.5020	0.5256	0.2762	0.369	on
Vaswani	11 429	24.8	93	0.4356	0.4571	0.1637	0.381	on
SciFact	5 183	145.5	300	0.6800	0.6832	0.6353	0.090	off
NFCorpus	3 633	156.7	323	0.3240	0.3307	0.1243	0.239	on
FiQA	57 638	71.6	648	0.2476	0.2624	0.1971	0.073	off
SciDocs	25 657	115.2	1 000	0.1568	0.1654	0.0966	0.087	off
ArguAna	8 674	96.5	1 406	0.3690	0.3538	0.2313	0.074	on
Touché-2020	382 545	155.3	49	0.4014	0.4899	0.1543	0.802	off
Quora	522 931	5.8	10 000	0.7858	0.7853	0.7376	0.119	off
Antique	403 666	20.4	200	0.3486	0.4033	0.2279	0.746	off
CORD-19 / TREC-COVID (dev)	171 332	109.7	50	0.6356	0.7458	0.0198	0.826	on

Across the twelve non-CORD collections the adapted engine beats textbook BM25 on nine, ties on Toy and effectively on Quora, and loses on one. That loss is ArguAna (0.3690 → 0.3538), and the cause is legible: its queries are whole arguments, so RM3 expands toward documents restating the same stance rather than the counterargument the qrels reward, and the document-as-query overlay catches the longest of these but not all. The largest gains, Touché (+0.089) and Antique (+0.055), both come from length-regime BM25 overrides with RM3 left off rather than from expansion.

3.1 Parameter search

I swept k_1 and b with `retrieve_bm25_only`, which skips RM3 and the query-length overlay. The sweep holds δ=0.80 and the title bonus at 0 so each curve isolates a single knob (Figure 1). On the 0.3-step grid nDCG rises monotonically with k_1 through 2.8 (0.6548), while b is essentially flat across $[0.5, 0.8]$ with a nominal peak of 0.6364 at b=0.70. A denser probe on the same run ($k_1 \in [2.0, 2.4]$) wobbles by at most 0.006: 2.10 → 0.6337, 2.15 → 0.6323, 2.18 → 0.6370. That is 50-topic noise, so the figure plots only the grid.

I treat ±0.01 nDCG@10 as the resolution of this dev set and do not ship the BM25-only argmax. High k_1 looks good on TREC-COVID: k_1=2.4 reaches 0.6440 on the first-pass grid, and 2.8 still higher. That is a 50-topic fit. With RM3 on, k_1=2.4 over-weights term frequency on these abstracts, and a single 2.4 point is the wrong BM25 for the other local collections (forums and short answers need a milder k_1). The shipped $(k_1, b) = (2.15, 0.60)$ on large abstracts, plus the shape-specific overrides, is what survived Table 4. From Table 2: the (k_1, b, δ) move is −0.003 (0.6356 → 0.6323), title β=0.60 is +0.049, then RM3 is +0.064.

4 Error analysis

The five TREC-COVID topics with the lowest nDCG@10 under `retrieve()` are below.

1. **Query 4** (0.066), **what causes death from Covid-19?** Question words are stopped, so the query is roughly {`caus`, `death`, `covid`}. Talk of death is common in this collection. The relevant documents are about ARDS and cytokine response, which share no stem with the query. RM3 then adds more death-related reviews. Bag-

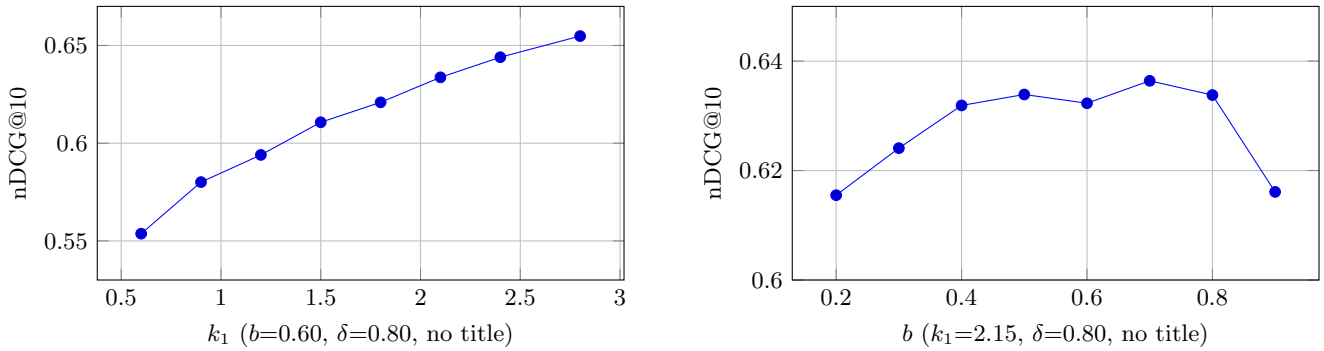


Figure 1: nDCG@10 on the 50 CORD-19 dev topics, BM25+ only, single process. The k_1 panel shows the 0.3-step grid. Off-grid, k_1=2.15 with b=0.60 gives 0.6323, the same as Table 2's "no title" row (and 0.001 below k_1=2.1, inside the ±0.01 band from a dense-probe wobble of ≤ 0.006). The shipped pair $(2.15, 0.60)$ is not the BM25-only peak, which is at k_1=2.8.

of-words has no path to the answer.

2. **Query 12** (0.191), **what are best practices in hospitals and at home in maintaining quarantine?** The title window boosts short overviews that start with `hospital` or `quarantine`. First-pass scores are all weak, not identical, so the variance guard does not fire.
3. **Query 31** (0.266), **How does the coronavirus differ from seasonal flu?** The topic asks for a difference, which a bag of words cannot state. There are no positions, so there is also no phrase match to use.
4. **Query 15** (0.366), **how long can the coronavirus live outside the body** This only partly matches the surface-stability words that the qrels use, and expansion copies that partial match.
5. **Query 16** (0.430), **how long does coronavirus remain stable on surfaces?** Closer wording than query 15, but still the fifth-worst topic: the first pass is only partly right, and RM3 keeps that mix.

RM3 copies what the first pass already thought: it helps when that pass is roughly right and hurts when it is confidently wrong. Two topics that look hard do well: query 42 (**Does Vitamin D impact COVID-19 prevention and treatment?**) reaches 0.948 because `vitamin` is rare once the lone `d` is dropped, and query 41 (**impacts of COVID-19 among African-Americans ...the rest of the U.S. population**) reaches 0.922 because `african` is selective and `U.S.` folds to `us`.

5 Final competition entry

The submitted `retrieve()` is C++ BM25+ with optional RM3. Knobs come from `adapt_params`(N, avgdl) and the query-length overlay in `engine.h`. It is not the Python BM25⊕VSM fusion. Boolean and VSM stay in the zip as tested components; they are not mixed into the ranking.

Tuning used the 50 public TREC-COVID topics. The collections in Table 4 were a check that those knobs still work on other collection shapes, not a second COVID fit. On the final course board this entry has nDCG@10 0.2284, MAP@10 0.1253, index 12,077,072 bytes on the held-out collection, efficiency +0.08, combined score 0.9233. Those nDCG/MAP numbers are the held-out topics; Table 2 is the public 50.

Reproducibility. `README.md` has the run sequence. The extension is built during `docker build`, not inside `build_index`. Worker count does not change the output bytes, gzip timestamps are 0, and ties break by score then doc id. Sorted JSON keys apply only to the Python `indexer.py` snapshot. Checks under `tests/`: retrievers (Boolean/VSM/BM25 vs hand-worked scores, k_1/b, ties); codecs (Simple-9/EF, acronyms); reproducibility (byte-identical rebuild, reload, doc-id ties, W=1 vs W=4, BM25-only skips query-length cuts); interface conformance (harness signatures); metrics (nDCG/MAP/MRR/P@k).

Code provenance

No search library is used in any submitted component. Porter stemming is my own C++ transcription of Porter (1980), not NLTK. The index stop list is the standard Lucene English list plus question words; the Fox list is used only to prune the RM3 forward index. Literature, as papers rather than source trees: Okapi BM25 (Robertson & Zaragoza, 2009); BM25+ (Lv & Zhai, 2011); RM3 (Lavrenko & Croft, 2001) with Dirichlet smoothing (Zhai & Lafferty, 2001). The newswire RM3 point (μ=1000, α=0.5, 10 docs / 10 terms) started from Anserini's published defaults and was re-tuned. Also Simple-9 (Anh & Moffat, 2005), Elias–Fano, front coding and vbyte (Manning et al.). Third party code in the tree is `pybind11` (build glue) and the starter harness.

GitHub link: `https://github.com/AdityaCodeSphere123/COL764-Assignment1/tree/student`